

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	3
ВВЕДЕНИЕ	8
1. Аналитический обзор методов ускоренной разметки и оценки достоверности ответов LLM	12
1.1. Задача оценки достоверности ответов LLM	12
1.2. Анализ существующих инструментов и подходов	12
1.2.1. Платформы разметки общего назначения	12
1.2.2. Подходы на основе LLM-ассистентов	13
1.2.3. Анализ задачи и формулировка требований	14
1.2.4. Сравнение подходов к автоматизации разметки (итог) ...	14
1.3. Методы дистилляции языковых моделей	14
1.4. Методы дообучения языковых моделей	15
1.5. Методы оценки качества языковых моделей	16
1.6. Выбор технологического стека	16
1.7. Защита данных, ограничения логов и риски применения LLM .	17
Выводы по главе 1	17
2. Проектирование инструмента анализа информационных контекстов	19
2.1. Общая архитектура системы	19
2.2. Конвейер сбора и предобработки данных	20
2.3. Конвейер генерации синтетического датасета	20
2.3.1. Назначение синтетического корпуса и риски	21
2.3.2. Типология, кластеры и квотирование	21
2.3.3. Промптинг генерации запросов	22
2.3.4. Промптинг и постобработка ответов учителя	22
2.3.5. Статистический контроль и версионирование	23
2.4. Конвейер обучения модели	23
2.5. Архитектура инференс-сервиса	24
2.6. Нефункциональные требования и эксплуатация сервиса	24
Выводы по главе 2	25
3. Реализация, тестирование и оптимизация инструмента разметки	27
3.1. Текущее состояние реализации	27
3.2. Конвейер сбора и предобработки данных	28

3.3. Генерация синтетического датасета	28
3.3.1. Волновая генерация запросов и квоты типов	28
3.3.2. Вызов учителя и постобработка ответов	29
3.3.3. Итоговый объём и разбиение	29
3.3.4. Статистическая верификация распределения	29
3.4. Эксперименты по дообучению и результаты	30
3.5. Стадия выравнивания методом CE RL	30
3.5.1. Описание функций вознаграждения	31
3.5.2. Группировка реворд-сигналов	32
3.5.3. Алгоритм подбора весов реворд-формулы и пороговая стабилизация	32
3.5.4. Поточечные метрики качества	35
3.6. Регрессионное тестирование, CI/CD и приёмочные критерии ..	35
3.7. Оптимизация инференса: квантизация FP8 и спекулятивное декодирование Eagle	36
3.7.1. Квантизация FP8	36
3.7.2. Спекулятивное декодирование Eagle	37
3.7.3. Сравнение Eagle-1 и Eagle-3	37
3.7.4. Методика замеров пропускной способности	37
3.7.5. Итоговая конфигурация и замеры	38
Выводы по главе 3	39
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	42
ПРИЛОЖЕНИЕ А. Диаграммы архитектуры	45
ПРИЛОЖЕНИЕ Б. Листинги ключевого кода	46
ПРИЛОЖЕНИЕ В. Скриншоты интерфейса платформы разметки	51

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

LLM (англ. *large language model*, большая языковая модель) — генеративная нейросетевая модель, обученная предсказывать следующий токен в тексте и применяемая для ответов на запросы, суммаризации, классификации и смежных задач. **Instruct-модель** — языковая модель, дополнительно обученная следовать инструкциям пользователя и отвечать в заданном формате.

Информационные контексты — релевантные фрагменты текстов из внешних источников (веб-документов), на которые опирается поисковая или диалоговая система при формировании ответа; в работе они служат опорой для проверки фактической согласованности ответа языковой модели.

Фактологическая подтверждённость — степень, в которой утверждение из ответа LLM подтверждается предоставленными информационными контекстами. В работе эта задача соответствует проверке того, можно ли считать фрагмент ответа подтверждённым, частично подтверждённым или неподтверждённым на основании документов.

Галлюцинация LLM — утверждение, сгенерированное языковой моделью, но не подтверждаемое исходными контекстами либо противоречащее им. Для задачи разметки такие ответы опасны, поскольку могут создавать видимость достоверного факта без опоры на источник.

Разметка — процесс ручной или полуавтоматической аннотации данных по заранее заданной схеме; **исполнитель** (разметчик) — специалист, выполняющий разметку в рамках платформы.

Reward-модель (модель вознаграждения) — обучаемая функция или нейросеть, оценивающая качество ответа (например, степень подтверждённости фрагмента текста информационными контекстами) числовым сигналом для обучения с подкреплением или ранжирования вариантов. **Reward-функция** — отдельное правило или модельная оценка, которая выдаёт численный сигнал по одному аспекту качества ответа: полноте, атрибуции, достоверности, фокусу и т.д.

Дистилляция знаний — перенос поведения крупной **модели-учителя** на меньшую по числу параметров **модель-ученика** путём обучения ученика на парах «вход — ответ учителя» и/или на дополнительных сигналах распределения вероятностей.

Дообучение — дополнительное обучение уже предобученной модели на предметно-специфичных данных. **Full fine-tune** — обновление всех весов модели; **LoRA / P-Tuning** — параметро-эффективные методы, при которых основная часть весов заморожена, а адаптация выполняется небольшим набором добавочных параметров. **SFT** (*supervised fine-tuning*) — дообучение модели на размеченных парах «вход — правильный ответ» с учителем. **Дистилляция на уровне ответов** (*response-based distillation*) — вариант дистилляции, при котором модель-ученик обучается повторять готовые ответы модели-учителя на заданных входах. **Модель только с декодером** (*decoder-only*) — архитектура трансформера, которая авторегрессионно генерирует текст последовательно слева направо по циклу. **Distillation from large models** — дистилляция от крупных моделей: подход, при котором большая модель-учитель формирует обучающие эталонные ответы или разметку, а меньшая модель-ученик обучается кросс-энтропийным методом на основе эталонных ответов.

Синтетический датасет — набор обучающих примеров, частично или полностью сгенерированных алгоритмами (в т.ч. LLM) по заданным правилам и ограничениям (искусственно моделируем заданное распределение). **Пайплайн** — последовательность автоматизированных шагов обработки данных или модели. **Чекпоинт модели** — сохранённое состояние модели после этапа обучения, которое можно использовать для инференса (в реальных сервисах) или дальнейшего дообучения.

LLM-as-a-judge — метод оценки качества ответов, при котором другая языковая модель выступает в роли «судьи», сравнивая варианты или выставляя баллы по формализованным критериям. **Поточечная оценка** (*pointwise*) — оценка одного ответа по шкале критериев без попарного сравнения с другим ответом.

CERL (*Cross Entropy Reinforcement Learning*) — подход к дообучению политики генерации с использованием кросс-энтропийной функции потерь и сигналов вознаграждения (в работе — стадия выравнивания после SFT). **Выравнивание модели** (*alignment*) — дополнительное обучение, направленное на согласование поведения модели с заданными критериями качества и ожиданиями пользователей.

Инференс — этап применения обученной модели для получения ответов на входные запросы (в отличие от этапа обучения). **Инференс-сервис** — программный компонент, предоставляющий доступ к модели, в работе — через **REST API**. **On-prem** — развёртывание сервиса внутри инфраструктуры организации, без передачи данных во внешние облачные API. **Production** — промышленная эксплуатационная среда, где сервис доступен пользователям и обслуживает реальные запросы.

API (*application programming interface*) — программный интерфейс взаимодействия компонентов. **REST API** — API, построенный вокруг HTTP-запросов и ресурсов. **JSON** — текстовый формат структурированных данных, используемый в работе для ответа модели и обмена данными между сервисами. **SLA** (*service level agreement*) — формализованные требования к эксплуатационным характеристикам сервиса. **Rate limiting** — ограничение числа запросов к сервису за единицу времени. **Confidence** — численная оценка уверенности модели или сервиса в сформированном ответе. **RPS** (*requests per second*) — число запросов к сервису в секунду. **Retry-After** — HTTP-заголовок, сообщающий клиенту, через сколько можно повторить запрос после какого-либо сбоя.

Precision, Recall, F-мера — метрики качества классификации на фиксированном наборе меток: точность (доля верно предсказанных положительных среди всех предсказанных положительных), полнота (доля найденных положительных среди всех истинно положительных); **F-мера** — их гармоническое среднее. **Базовый вариант** — исходное решение или модель, с которыми сравниваются новые модели или конфигурации. **Regex-метрики** — автоматические проверки на основе регулярных выражений, которые контролируют наличие нужных полей, формат ответа и другие структурные признаки.

Квантизация FP8 — представление весов и/или активаций в формате с плавающей запятой из 8 бит для ускорения вычислений и снижения объёма памяти. **BF16** — формат чисел с плавающей запятой bfloat16, часто используемый для обучения и инференса нейросетей. **GPU** — графический процессор, используемый для ускорения вычислений модели. **VRAM** — видеопамять GPU, в которой размещаются веса модели, KV-кэш и рабочие буферы. **KV-кэш** — память ключей и значений механизма внимания трансформера, которая хранится при генерации и ускоряет продолжение уже начатого от-

вета. **Батч** — группа запросов или обучающих примеров, обрабатываемых моделью за один шаг.

Спекулятивное декодирование — ускорение генерации за счёт черновых предсказаний небольшой моделью с последующей верификацией большой моделью; в работе рассматривается семейство методов **Eagle**. **TTFT** (*time to first token*) — время от поступления запроса до генерации первого токена ответа. **p95** и **p99** — 95-й и 99-й перцентили задержки: значения, ниже которых укладываются соответственно 95% и 99% запросов.

Zero-shot — режим применения модели без примеров решения конкретной задачи в промпте. **Top- p** — параметр генерации, при котором модель выбирает следующий токен из минимального набора наиболее вероятных токенов с суммарной вероятностью не ниже p . **Промпт** — текстовая инструкция и входные данные, передаваемые языковой модели. **Температура генерации** — параметр, управляющий степенью случайности выбора токенов. **Top- k** — ограничение выбора следующего токена только k наиболее вероятными вариантами. **Logits** — необработанные числовые оценки вероятностей токенов до применения нормализации softmax.

YTSaurus — распределённая система хранения и обработки данных, используемая для логов, датасетов и промежуточных таблиц. **MinHash-LSH** — метод приближённого поиска похожих текстов: MinHash оценивает сходство множеств токенов, а LSH ускоряет поиск почти дубликатов. **PII** (*personally identifiable information*) — персональные данные или сведения, позволяющие идентифицировать человека. **Индекс Жаккара** — мера сходства двух множеств, равная отношению размера их пересечения к размеру объединения.

NLP — обработка естественного языка; **CV** — компьютерное зрение. **Crowdsourcing-платформа** — платформа, распределяющая задачи между большим числом исполнителей. **ML-бэкенд** — серверная часть системы, на которой развернута модель машинного. **Labeling function** — функция, дающая по данным метку.

HTTP — протокол прикладного уровня для передачи запросов между клиентом и сервером. **Stateless-сервис** — сервис, не хранящий пользовательское состояние между запросами. **TTL** (*time to live*) — срок хранения данных или артефакта, после которого он удаляется автоматически.

CI/CD — автоматическая сборка, тестирование и выкатка новых версий. **Docker** — технология упаковки приложения в контейнер с зависимостями. **Канареечная выкатка** — постепенный запуск новой версии на небольшой доле трафика перед полным развёртыванием. **A/B-сравнение** — эксперимент, в котором две версии системы сравниваются на сопоставимых группах запросов или пользователей. **wrk** и **Locust** — инструменты нагрузочного тестирования HTTP-сервисов.

ZeRO — метод оптимизации памяти при обучении больших моделей за счёт распределения параметров, градиентов и состояний оптимизатора. **Flash-Attention** — эффективная реализация механизма внимания трансформера, уменьшающая потребление памяти и ускоряющая обучение или инференс. **Модель-критик** — дополнительная модель в алгоритмах обучения с подкреплением, оценивающая действия основной модели; в данной работе отдельная модель-критик не используется. **Hugging Face Transformers** — библиотека для работы с современными нейросетевыми языковыми моделями. **PEFT** — семейство параметро-эффективных методов дообучения, к которому относится LoRA.

ВВЕДЕНИЕ

Современные генеративные языковые модели (LLM) находят широкое применение в интеллектуальных поисковых системах и голосовых ассистентах. Система Alice AI LLM Search компании «Яндекс» формирует ответы на поисковые запросы пользователей, опираясь на *информационные контексты* — релевантные фрагменты текстов, извлечённые из интернет-источников. Качество генерируемых ответов напрямую определяет пользовательский опыт, поэтому обеспечение их фактической достоверности является приоритетной задачей [5]. **Актуальность** темы опирается на обзоры масштабирования и ограничений LLM [12, 18, 19], работы по обучению с подкреплением и выравниванию поведения моделей [6, 23, 28], методам параметро-эффективного дообучения [7, 20], обобщающим исследованиям дистилляции [8, 16, 22], подходам извлечения опоры на документах и оценке согласованности с контекстом [1, 24, 26], обзорам инструментов и метрик разметки [2, 14, 25], методу LLM-as-a-judge [17], инженерным приёмам ускорения обучения и инференса [4, 11, 13, 29], квантизации вычислений [21] и спекулятивному декодированию [10].

Для обучения *reward-модели*, оценивающей степень подтверждённости ответа LLM информационными контекстами, требуется большой объём размеченных данных. Разметчики вручную проверяют каждый фрагмент текста ответа по трёхклассовой шкале: **0** — не подтверждён, **1** — частично подтверждён, **2** — подтверждён. Текущий процесс занимает в среднем 7–8 минут на задание, из которых 3–4 минуты на задание расходуется на поиск релевантных документов в информационных контекстах и ещё 4 минуты на задание — на верификацию фактов. Подобная трудоёмкость делает разметку «бутылочным горлышком» в конвейере подготовки данных [14].

Для частичной автоматизации поиска контекстов текущий пайплайн использует модель DeepSeek-V3 в качестве внешнего LLM-ассистента. Однако этот подход имеет существенные недостатки: время генерации ответа составляет 2–5 минут на 1 фрагмент из задания, применяются неструктурированные промпты, а качество ответов удовлетворяет исполнителей лишь в 50% случаев. Ключевые метрики текущей разметки (F-мера $\approx 0,60$, Precision $\approx 0,55$, Recall $\approx 0,57$) не отвечают требованиям к обучающим данным для production-моделей [25].

Для устранения указанных проблем предлагается разработать **специализированный инструмент** на основе дистиллированной языковой модели [16]. Дистиллированная модель меньших размеров (7–32 млрд параметров вместо ~ 600 млрд у DeepSeek-V3), обученная непосредственно на задаче анализа информационных контекстов, обеспечит значительно меньшее время инференса и более высокое качество ответов благодаря узкоспециализированному дообучению.

Цель работы — улучшить систему разметки фрагментов, ускорив процесс **в два раза** до 3–4 минут на задание (сократив время поиска контекстов вдвое), повысив точность до F-меры $> 0,85$ (с текущих $\approx 0,60$), обеспечив быстрый отклик LLM с латентностью ≤ 30 секунд на 1 фрагмент из задания (вместо 2–5 минут на 1 фрагмент из задания) при помощи **спекулятивного декодирования** (алгоритм Eagle) и **квантизации в FP8**, минимизировав галлюцинации (с $\approx 30\%$) и подняв удовлетворённость команды выше 70% (с $\approx 50\%$), сделав её быстрой, точной, надёжной и удобной.

Задачи:

- а) Подобрать архитектуру и размер генеративной модели-«ученика» с балансом скорость/качество и спроектировать процедуру её обучения.
- б) Сформировать синтетические датасеты запросов и ответов (distillation from large models), реализовать очистку/фильтрацию и контроль распределений.
- в) Реализовать пайплайны оркестрации (статический и динамический граф) для подготовки данных, генерации, обучения, оценки и инференса.
- г) Разработать метрики автоматической оценки качества ответов (правила/регулярные выражения + LLM-as-a-judge) и провести серию экспериментов по гиперпараметрам и оптимизациям обучения/инференса.

Объект исследования — процесс текстовой разметки ответов генеративной языковой модели Alice AI LLM Search по критерию достоверности фактов.

Предмет исследования — методы и алгоритмы автоматического анализа информационных контекстов для ускорения и повышения качества текстовой разметки.

Новизна работы. В отличие от существующих решений (универсальные LLM-API, платформы разметки, DeepSeek-V3 без дообучения), предлагаемый подход сочетает три элемента, ранее не применявшихся совместно в задаче ассистирования разметчиков:

- а) *Специализированная дистилляция* ответов учителя (DeepSeek-V3.1-Terminus) на корпусе реальных доменных запросов исполнителей с балансировкой по типам запросов;
- б) *Стадия выравнивания методом CE RL* с шестью факторными функциями вознаграждения (атрибуция, полнота, достоверность, ясность, цитаты, фокус), оцениваемыми LLM-судьёй, и их групповой агрегацией;
- в) *On-prem инференс* специализированной модели (YaGPT 7b) со структурированным выходом, встроенной в интерфейс платформы разметки — без передачи данных во внешние API.

Совокупность этих элементов позволяет впервые достичь латентности ≤ 30 с на 1 фрагмент из задания при F-мере $> 0,85$ в задаче разметки фактологической подтверждённости в закрытой производственной инфраструктуре.

Практическая значимость. Инструмент внедрён в промышленную платформу текстовой разметки ООО «Яндекс.Технологии» в виде диалогового окна и доступен через REST API. Ожидаемый измеримый эффект: ускорение разметки в 2 раза (до 3–4 мин на задание), F-мера выше 0,85, латентность ответа модели ≤ 30 с на 1 фрагмент из задания, рост удовлетворённости исполнителей выше 70%, снижение доли несогласованных с контекстом ответов по сравнению с универсальной внешней LLM. Внедрение подтверждается актом о внедрении от ООО «Яндекс.Технологии». Результаты работы представлены на XV Конгрессе молодых учёных; материалы планируются к публикации на конференции AINL 2026.

Структура работы. В первой главе приведён аналитический обзор методов ускоренной разметки и оценки достоверности ответов LLM: рассмотрены существующие платформы разметки, подходы на основе LLM-ассистентов, методы дистилляции и дообучения, метрики качества и выбор технологического стека; сформулированы требования и постановка задачи. Во второй главе описано проектирование инструмента анализа информационных контекстов — общая архитектура (C4), конвейеры сбора данных, генерации датасета и обучения, архитектура инференс-сервиса, логическая мо-

дель данных. В третьей главе приведены реализация, тестирование и оптимизация инструмента: конвейеры, синтетический датасет, эксперименты по дообучению и стадия CE RL, регрессионные и нагрузочные проверки, результаты внедрения, а также инженерная оптимизация инференс-сервиса (квантизация FP8, спекулятивное декодирование Eagle), методика замеров и сравнение конфигураций.

ГЛАВА 1. АНАЛИТИЧЕСКИЙ ОБЗОР МЕТОДОВ УСКОРЕННОЙ РАЗМЕТКИ И ОЦЕНКИ ДОСТОВЕРНОСТИ ОТВЕТОВ LLM

1.1. Задача оценки достоверности ответов LLM

С ростом применения LLM в поисковых системах остро встала проблема *галлюцинаций* — генерации фактически неверной информации [12, 18]. Для контроля достоверности ответов системы Alice AI LLM Search применяется отдельная reward-модель, оценивающая степень подтверждённости каждого фрагмента ответа информационными контекстами [1, 26]. Обучение такой модели требует сотен тысяч размеченных образцов.

Разметка каждого образца — это задача *многоклассовой классификации*: разметчик изучает фрагмент текста ответа LLM и список информационных контекстов, а затем присваивает класс: **0** — факт не подтверждён ни одним контекстом, **1** — факт частично подтверждён ($> 50\%$ сходимости), **2** — факт полностью подтверждён. Ключевая трудоёмкость операции — поиск среди десятков документов именно тех фрагментов, которые подтверждают или опровергают конкретный факт.

1.2. Анализ существующих инструментов и подходов

1.2.1. Платформы разметки общего назначения

На рынке существует ряд зрелых платформ разметки данных: Label Studio, Argilla, Prodigy, Scale AI Nucleus, Toloka (Яндекс). Все они предоставляют пользовательский интерфейс разметчика и инструменты контроля качества, однако принципиально не решают проблему поиска релевантных фрагментов в контекстах. Разметчик по-прежнему просматривает документы вручную.

Label Studio [15] — открытая платформа с поддержкой широкого спектра типов разметки (NLP, CV, audio). Позволяет добавить ML-бэкенд для пре-аннотации, однако не содержит специализированных компонентов для задачи оценки фактологической подтверждённости: поиска фрагментов документа, подтверждающих или опровергающих конкретное утверждение.

Argilla [3] — платформа, ориентированная на сбор обратной связи для RLHF. Поддерживает обратную связь на уровне ответов модели, но не даёт разметчику инструмента для гранулярной работы с контекстами: нет механизма выделить конкретный фрагмент-источник и сопоставить его с утверждением в ответе LLM.

Snorkel AI [27] предлагает программируемую разметку (labeling functions) и слабый надзор. Этот подход эффективен для задач с чёткими паттернами, однако верификация фактических утверждений в ответах LLM требует семантического понимания, которое не формализуется детерминированными правилами.

Scale AI / Toloka — crowdsourcing-платформы. Масштабируют ручной труд, но не сокращают время на одного исполнителя и сохраняют требование к предметной экспертизе.

Вывод: ни одна из платформ не предоставляет специализированного ассистента, который бы *автоматически* находил и атрибутировал подтверждающие фрагменты из контекстов. Расширение любой из них до требуемой функциональности потребовало бы разработки того же ML-компонента, что и в данной работе, плюс дополнительной интеграции в закрытую инфраструктуру Яндекса.

1.2.2. Подходы на основе LLM-ассистентов

Второй класс решений — использование общей инструктируемой LLM в качестве ассистента разметчика. В таблице 1 приведено сравнение по ключевым параметрам задачи.

Таблица 1 – Сравнение LLM-ассистентов для разметки фактологической подтверждённости

Решение	Латент-ность	F-мера
GPT-4o (API)	5–15 с	$\sim 0,72^*$
Claude 3.5 Sonnet (API)	8–20 с	$\sim 0,70^*$
DeepSeek-V3 (текущий)	2–5 мин на 1 фрагмент из задания	0,60
YaGPT 7b zero-shot	20–40 с	0,51
YaGPT 7b + SFT + CE RL	≤ 30 с на 1 фрагмент из задания	$> 0,85$

* Оценка по аналогичным задачам fact-checking из [2, 14]; прямое тестирование на закрытых данных не проводилось.

Публичные API (GPT-4o, Claude) недоступны как on-prem решения: данные разметчиков содержат внутреннюю информацию компании. DeepSeek-V3 в текущей конфигурации работает без специализированного промпта и дообучения — отсюда 2–5 минут на 1 фрагмент из задания и F-мера = 0,60. YaGPT 7b в режиме zero-shot даёт приемлемую латентность, но

не умеет возвращать структурированный ответ с атрибуцией и цитированием: $F\text{-мера} = 0,51$.

Ключевой вывод: разрыв между zero-shot YaGPT 7b (0,51) и целевым уровнем ($> 0,85$) не закрывается промпт-инжинирингом — модель не видела доменных примеров и не умеет формировать структурированные ответы с атрибуцией. Дообучение на задаче является необходимым и единственным on-prem-совместимым путём.

1.2.3. Анализ задачи и формулировка требований

На основании сравнения определены ключевые требования к разрабатываемому инструменту:

- а) **Латентность** ≤ 30 с на **1 фрагмент из задания** (текущий узкий участок — 2–5 мин на 1 фрагмент из задания);
- б) **F-мера** $> 0,85$ при трёхклассовой классификации достоверности;
- в) **On-prem деплой:** модель работает на GPU Яндекса без передачи данных во внешние API;
- г) **Структурированный ответ** с атрибуцией к конкретным фрагментам контекстов (attribution, quotes);
- д) **Специализация:** дообучение на запросах исполнителей платформы разметки, а не на общем корпусе.

Ни одно из рассмотренных решений не удовлетворяет всем пяти требованиям одновременно. Данная работа закрывает этот пробел.

1.2.4. Сравнение подходов к автоматизации разметки (итог)

В таблице 2 сведено итоговое сравнение по классам подходов.

1.3. Методы дистилляции языковых моделей

Дистилляция знаний [16] — метод переноса знаний из большой модели-учителя в меньшую модель-ученика. В классическом варианте ученик обучается воспроизводить «мягкие» вероятностные распределения учителя, а не жёсткие метки классов, что позволяет передать дополнительную информацию о межклассовых отношениях.

Для языковых моделей распространена **дистилляция на уровне ответов** (*response-based distillation*): ученик обучается на корпусе вопрос-ответных пар, где ответы сгенерированы учителем [7]. Этот подход особенно

Таблица 2 – Сравнение классов подходов к автоматизации разметки

Подход	Скорость	Качество
Ручная разметка	7–8 мин на задание	Высокое
Платформы (Label Studio и др.)	7–8 мин на задание	Высокое
Универсальный LLM-API	5–20 с	Среднее
DeerSeek-V3 без FT (текущий)	2–5 мин на 1 фрагмент из задания	Низкое
YaGPT 7b + SFT + CE RL	≤ 30 с на 1 фрагмент из задания	Высокое

эффективен, когда имеется доступ к сильному учителю, но не к его параметрам (black-box distillation) [8].

В данной работе применяется метод **Distillation from Large Models**: модель DeerSeek-V3.1-Terminus генерирует эталонные ответы на запросы исполнителей, которые затем используются для дообучения модели-ученика (YaGPT 7b/8b/32b). Основные этапы: сбор промптов → генерация ответов учителем → пост-обработка (фильтрация, балансировка) → обучение ученика.

1.4. Методы дообучения языковых моделей

Full fine-tuning предполагает обновление всех параметров модели. Позволяет добиться максимальной специализации, однако требует значительного объёма GPU-памяти и большого датасета ($\geq 10,000$ образцов). Используется в начальных экспериментах на синтетическом датасете.

LoRA (Low-Rank Adaptation) [20] — эффективный метод параметрического дообучения: добавляются низкоранговые адаптерные матрицы к весам трансформера, обучаются только они (0,1–1% параметров). Применяется для дообучения на реальных запросах исполнителей ($< 1,500$ образцов).

RLHF / CERL (Cross Entropy Reinforcement Learning) [23, 28] — метод обучения с подкреплением, где reward-сигнал поступает от LLM-судьи. Планируется применить после получения базового чекпоинта для улучшения качества структурированности ответов.

DPO (Direct Preference Optimization) [6] — альтернатива RLHF без явного обучения reward-модели. Рассматривается как запасная стратегия при нестабильности CERL.

1.5. Методы оценки качества языковых моделей

Традиционные метрики (BLEU, ROUGE) плохо коррелируют с человеческой оценкой для задач генерации структурированных текстов [2]. В работе используется комбинация двух подходов.

Regex-метрики — проверка соответствия ответа модели-ученика заданному формату с помощью регулярных выражений. Оценивают корректность структуры ответа: наличие обязательных полей, формат ссылок на источники, корректность классификационных меток. Работают быстро, детерминированы.

LLM-as-a-judge [17] — оценка качества ответа с помощью более мощной модели (DeepSeek-V3.1-Terminus). Промпт содержит ответ учителя, ответ ученика и критерии оценки: точность фактов, глубина анализа, наличие галлюцинаций, соответствие контексту. Позволяет оценить семантическое качество ответов.

1.6. Выбор технологического стека

Выбор технологий обусловлен производственными ограничениями ООО «Яндекс.Технологии» и требованиями к производительности.

PyTorch — основной фреймворк машинного обучения. Выбран за развитую экосистему (Hugging Face Transformers, PEFT), поддержку Flash-Attention [4, 13] и ZeRO-оптимизации [29] для эффективного обучения на GPU Nvidia H100.

Внутренний аналог Apache Airflow (статический) — платформа оркестрации конвейеров с фиксированным графом задач. Используется для сбора данных, генерации датасета и обучения модели.

Внутренний аналог Apache Airflow (динамический) — платформа исполнения конвейеров в реальном времени с динамическим графом узлов. Используется для обработки запросов в production.

YTSaurus — платформа распределённого хранения и обработки данных с открытым исходным кодом. Используется для хранения логов, датасетов и артефактов обучения.

Внутренний аналог vLLM [11] — фреймворк ускоренного инференса с PagedAttention. Используется для деплоя обученной модели в production.

NLTK — используется для предобработки текстовых данных: токенизация, стемминг, анализ распределения токенов.

1.7. Защита данных, ограничения логов и риски применения LLM

Конвейер обучения и инференса опирается на логи запросов исполнителей и фрагменты ответов LLM. Для снижения юридических и репутационных рисков приняты следующие принципы. **Минимизация данных:** в таблицы YTSaurus попадают только поля, необходимые для воспроизведения промпта и оценки качества (идентификатор задания, тип запроса, нормализованный текст, ссылки на контексты без полного HTML-страниц, где это возможно). **Псевдонимизация:** идентификаторы исполнителей заменяются на устойчивые хэши; прямые ФИО и контактные данные в обучающих выборках не хранятся. **Сроки хранения:** сырые логи и промежуточные артефакты пайплайна удаляются по политике TTL оркестратора; чекпоинты и финальные датасеты версионизируются и доступны ограниченному кругу сервисных аккаунтов.

Риски использования LLM в цикле разметки включают: утечку чувствительных формулировок из контекстов в логи ошибок инференса; «запоминание» редких шаблонов из обучающей выборки (меморизация); смещение распределения ответов при дообучении на узком корпусе. Противодействие: on-prem деплой без внешних API, фильтрация PII на этапе предобработки, регулярный пересмотр выборки и контроль метрик на отложенной выборке, не участвовавшей в подборе гиперпараметров.

Выводы по главе 1

В результате аналитического обзора установлено, что основная проблема действующего процесса разметки связана не только с собственно принятием решения по метке достоверности, но и с длительным поиском подтверждающих контекстов, проверкой противоречий и ожиданием ответа универсальной внешней LLM. Существующие платформы разметки общего назначения (Label Studio, Argilla, Snorkel AI) хорошо закрывают задачи организации рабочего процесса, но не решают доменную задачу оценки фактологической подтверждённости для фрагментов ответов LLM без дополнительного интеллектуального ассистента. Подход с использованием крупной внешней модели без дообучения также недостаточен: он даёт высокую латентность, не гарантирует стабильный структурированный формат ответа и не обеспечивает требуемую F-меру выше 0,85.

Анализ методов показал, что наиболее обоснованным направлением является разработка специализированной дистиллированной модели: крупная модель-учитель формирует эталонные ответы, а компактная модель-ученик обучается на целевом домене и затем разворачивается on-prem внутри производственного контура. Для достижения требуемого качества целесообразно сочетать несколько этапов: Full fine-tune на синтетическом корпусе, параметро-эффективное дообучение LoRA на реальных запросах и последующую стадию CE RL для выравнивания структуры, полноты, атрибуции и фокуса ответов. Такая схема лучше соответствует ограничениям задачи, чем только настройка промптов или использование универсального API.

В качестве методов контроля качества выбрана комбинация детерминированных regex-проверок и оценки LLM-as-a-judge, поскольку одна группа метрик проверяет структурированный ответ генеративной модели, заданный в чётком формате при помощи системного промпта, а другая позволяет оценить семантические свойства: достоверность, полноту, атрибуцию, качество цитирования и фокус. Выбранный технологический стек (PyTorch, YTSaurus, внутренний аналог vLLM, оркестраторы статического и динамического графа) согласуется с производственными требованиями к воспроизводимости, хранению данных, скорости инференса и изоляции чувствительных контекстов. Тем самым в первой главе обоснована постановка задачи, целевые метрики и необходимость проектирования специализированной системы, а не отдельного экспериментального скрипта или разовой интеграции с внешней LLM.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ИНСТРУМЕНТА АНАЛИЗА ИНФОРМАЦИОННЫХ КОНТЕКСТОВ

2.1. Общая архитектура системы

На рисунке 1 приведена контейнерная диаграмма системы (C4 Level 2 — Container): границы Annotation Assistant, основные контейнеры и потоки данных относительно исполнителя и платформы разметки; красным выделен вклад автора. Инструмент анализа информационных контекстов выступает промежуточным звеном между исполнителем и платформой разметки: принимает запрос исполнителя и информационные контексты, возвращает структурированный анализ достоверности.

Диаграммы в нотациях C4/UML и логическая модель данных.

Для проектной документации используются диаграммы **C4 Container** (рис. 1), **UML Sequence** для сценария вызова ассистента (рис. 2), а также блок-схемы конвейеров на рисунках 3–5, отражающие потоки данных между хранилищами и сервисами и выполняющие роль упрощённых **DFD**. Полное BPMN-описание допускается как расширение тех же потоков при согласовании с заказчиком; в тексте ВКР достаточно показанной декомпозиции этапов.

На концептуальном уровне (**ER**) выделяются сущности: *Исполнитель*, *Задание разметки*, *Информационный контекст*, *Запрос к ассистенту*, *Структурированный ответ модели*, *Запись обучающего датасета*, *Версия чекпоинта*. Исполнитель выполняет множество заданий; задание объединяет множество контекстов; запрос к ассистенту порождает один ответ с атрибуцией; записи датасета агрегируются из логов и ответов учителя; чекпоинт версионизируется и связывается с конфигурацией сервиса. Физическое отображение — таблицы и каталоги YTSaurus, объектное хранилище артефактов (см. рис. 1, приложение А).

На рисунке 2 представлена диаграмма последовательности запроса анализа информационных контекстов: от действия исполнителя до получения структурированного ответа (класс достоверности, атрибуция, цитаты).

Компонентная диаграмма конвейера обучения модели приведена в Приложении А.

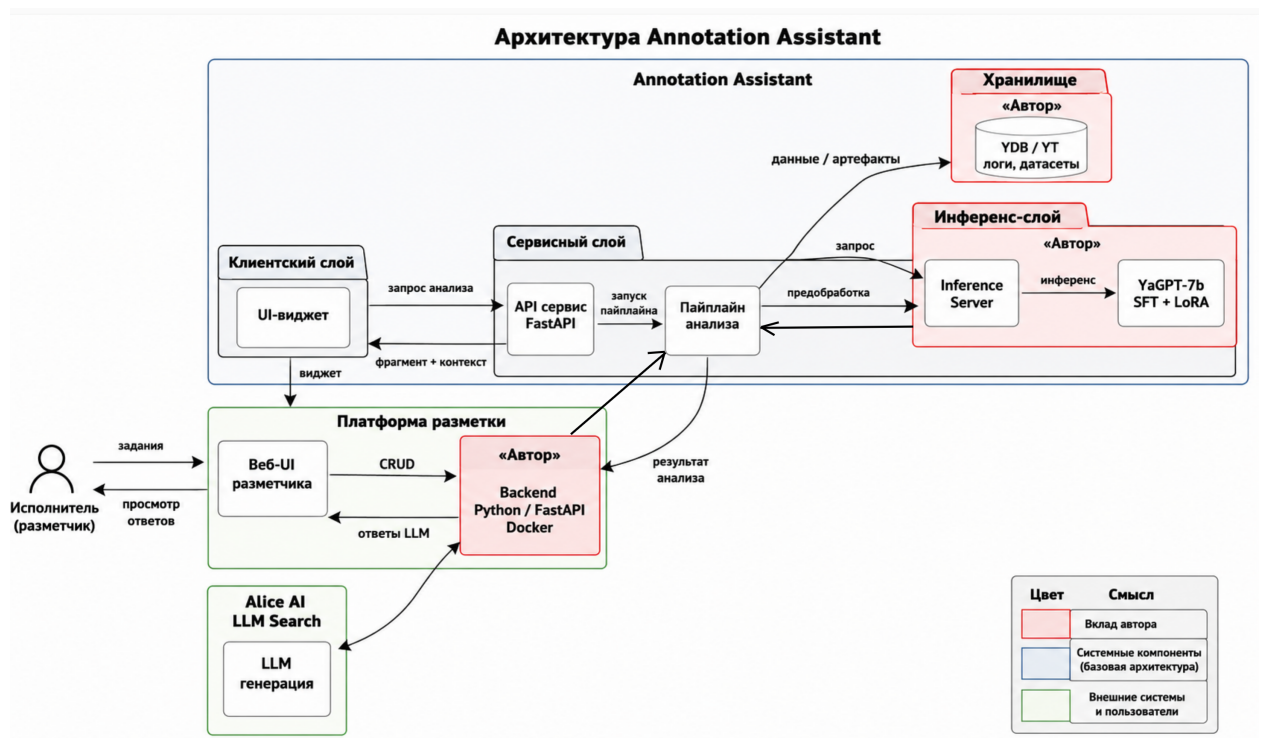


Рисунок 1 – Контейнерная диаграмма системы (C4 Level 2 — Container). Красным выделен вклад автора

2.2. Конвейер сбора и предобработки данных

Конвейер реализован в виде последовательной цепочки узлов на платформе статического оркестратора (рисунок 3). Входными данными служат логи платформы разметки, хранящиеся в YTSaurus.

Узел **«Очистка и нормализация»**: удаление дублирующих пробелов, html-тегов, нормализация unicode, отфильтровка слишком коротких запросов (< 10 токенов) и нетематических сообщений.

Узел **«Дедупликация»**: удаление точных и нечётких дубликатов (с порогом Jaccard $\geq 0,9$) с помощью MinHash LSH.

Узел **«Анализ типов запросов»**: кластеризация запросов (K-means на TF-IDF эмбедингах) для последующего контроля равномерности распределения в синтетическом датасете.

2.3. Конвейер генерации синтетического датасета

Конвейер генерации (рисунок 4) состоит из двух логических ветвей: синтез доменных запросов по типологиям реальных логов и получение эталонных ответов моделью-учителем с последующей фильтрацией. Ниже зафиксированы принятые проектные решения; реализация и численные итоги приведены в главе 3.

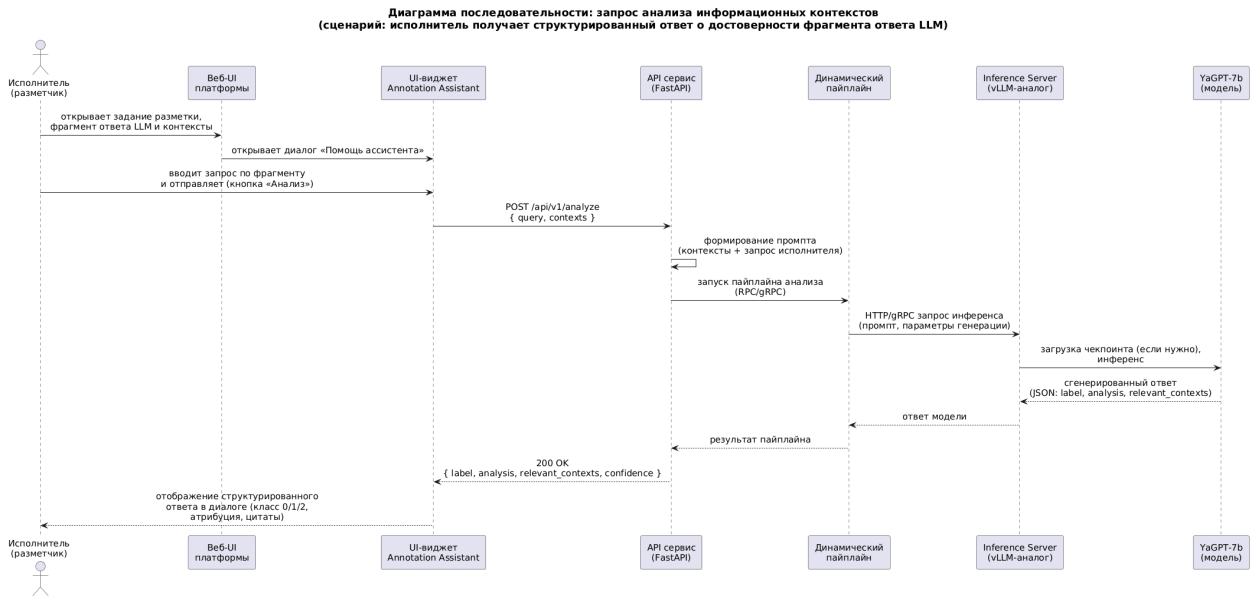


Рисунок 2 – Диаграмма последовательности: запрос анализа информационных контекстов

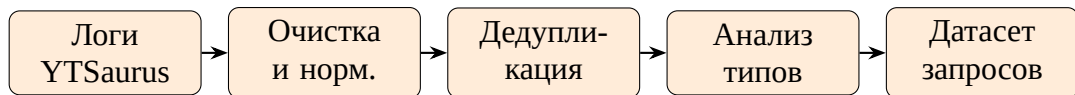


Рисунок 3 – Конвейер сбора и предобработки данных

2.3.1. Назначение синтетического корпуса и риски

Пары «запрос исполнителя — эталонный разбор контекстов» необходимы, чтобы в supervised-фазе задать объём и стиль поведения модели-ученика в домене платформы разметки при ограниченном числе вручную размеченных экземпляров [14, 25]. Согласно парадигме дистилляции на уровне ответов учитель (DeepSeek-V3.1-Terminus) задаёт целевые ответы на масштабе, недоступном ручной разметке [16]. Риск смещения распределения при чисто синтетическом обучении учитывается: после SFT на синтетике выполняется дообучение LoRA на реальных логах и стадия выравнивания, что согласуется с рекомендациями по оценке и смешению источников данных для LLM [2].

2.3.2. Типология, кластеры и квотирование

Опорными остаются четыре типа запросов, выделенные по логам (см. перечисление в главе 3). На этапе проектирования зафиксировано:

- для каждого типа в few-shot шаблон включаются 3–5 репрезентативных примеров из соответствующих кластеров K-means на TF-IDF (согласованно с узлом «Анализ типов» конвейера сбора);

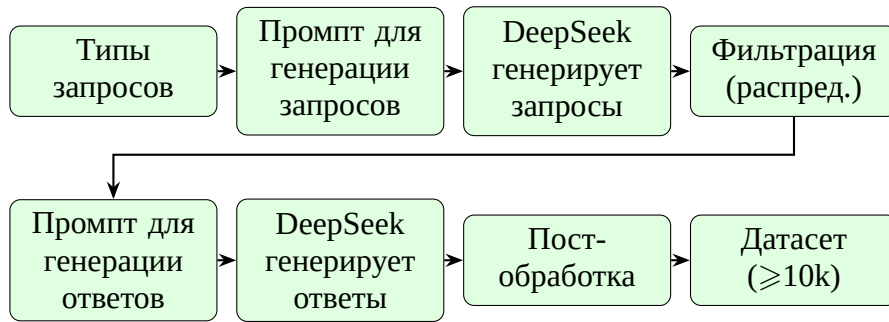


Рисунок 4 – Конвейер генерации синтетического датасета

- целевые доли типов в финальном корпусе задаются по эмпирическим частотам после дедупликации логов;
- генерация организуется **стратифицированно**: на каждой итерации пересчитывается текущий дефицит типа относительно квоты, и объём запросов следующей волны распределяется пропорционально дефициту. Отбраковываются дословные и почти дословные повторы few-shot примеров, а также кандидаты ниже порога по числу токенов (согласованного с порогом ≥ 10 токенов на этапе очистки логов).

2.3.3. Промптинг генерации запросов

Инструкция к модели-генератору запросов формулируется как задача «синтезировать новые формулировки той же семантической цели», без копирования лексики примеров; допускается вариация длины и наличие уточняющих подвопросов внутри типа. Параметры декодирования на стороне генератора запросов (температура, *top-p*, лимит выхода) подбираются эмпирически на отложенной мини-выборке: цель — разнообразие формулировок при сохранении доменной принадлежности к задаче оценки фактологической подтверждённости.

2.3.4. Промптинг и постобработка ответов учителя

Промпт к DeepSeek-V3.1-Terminus строится в формате, близком к production-разметке: **zero-shot** относительно конкретной пары «запрос — контексты», но с полностью зафиксированной схемой выхода. Целевая структура включает поля `relevant_contexts`, `analysis`, `conclusion`, `label` (в промышленном структурированном формате, эквивалентном листингу Б.2). Постобработка проектируется как конвейер проверок:

- разбор структурированного ответа и отсеечение ответов вне схемы;

- проверка индексов контекстов на принадлежность диапазону;
- regex- и эвристические фильтры против пустого анализа, несоответствия числу ссылок и заведомо неконсистентных меток;
- контроль баланса классов 0/1/2 на уровне батча (при перекосе следующие вызовы учителя приоритизируют недостающий класс).

2.3.5. Статистический контроль и версионирование

Согласование наблюдаемых долей типов запросов в синтетическом корпусе с долями в предобработанных логах проверяется χ^2 -критерием на уровне значимости 0,05; при отвержении гипотезы планируется добор генерации по недопредставленным типам либо корректировка квот. Каждая выпущенная версия корпуса сопровождается в метаданных YTSaurus идентификатором запуска пайплайна, версией промптов и параметров декодирования учителя, чтобы обеспечить воспроизводимость экспериментов по обучению и аудит регрессий качества.

2.4. Конвейер обучения модели

На рисунке 5 представлена схема конвейера обучения, реализованного на статическом оркестраторе.

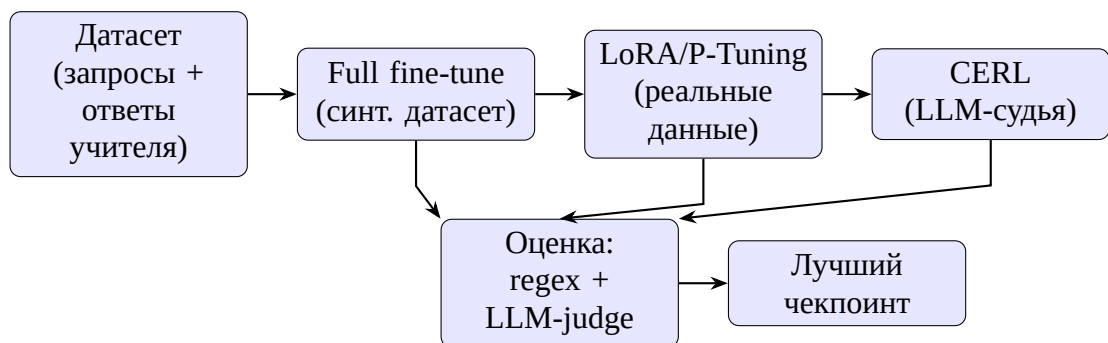


Рисунок 5 – Конвейер обучения модели

Сетка гиперпараметров:

- Число эпох: 3, 5, 10;
- Learning rate: 10^{-6} , 10^{-5} , 10^{-4} ;
- Стратегии scheduler: cosine_annealing, reduce_on_plateau;
- Оптимизации инфраструктуры: Flash-Attention 2/3 [4], ZeRO-2/3 [29], gradient checkpointing;
- GPU: Nvidia H100 80GB, количество определяется бенчмарком.

2.5. Архитектура инференс-сервиса

Сервис развёрнут на основе внутреннего аналога vLLM [11] с PagedAttention для эффективного управления KV-кэшем. Запрос исполнителя поступает в динамический пайплайн оркестратора, где составляется промпт (информационные контексты + вопрос исполнителя) и отправляется в инференс-сервер. Ответ возвращается в структурированном формате, заданном системным промптом, и отображается в диалоговом окне платформы разметки.

REST API сервиса:

- `POST /api/v1/analyze` — основной эндпоинт;
- Тело запроса: `{query: str, contexts: list[str]}`;
- Тело ответа: `{label: int, analysis: str, relevant_contexts: list[int], confidence: float}`;
- Целевое время ответа: ≤ 30 сек на 1 фрагмент из задания.

2.6. Нефункциональные требования и эксплуатация сервиса

Помимо функциональной спецификации API зафиксированы **SLA** для production: доступность сервиса анализа не ниже 99,5% в месяц (исключая плановые окна обновления модели не более двух часов в квартал); p95 задержки ответа при типичной длине контекста (до 32 000 токенов в сумме по полям) не превышает 35 сек; p99 — не более 45 сек. При превышении порогов включается эскалация в дежурную смену платформы разметки.

Пропускная способность и очереди. API защищён rate limiting по ключу платформы и по пользователю: не более N параллельных запросов на пул GPU, очередь отклоняется с кодом HTTP 429 и заголовком `Retry-After`. Для длинных запросов поддерживается асинхронный режим (идентификатор задачи + опрос статуса), чтобы не блокировать воркеры веб-интерфейса.

Наблюдаемость. Каждый запрос получает `trace_id`; в структурированные логи пишутся длительность этапов пайплайна (сборка промпта, вызов инференса, постобработка структурированного ответа), код ответа модели, число принятых токенов при спекулятивном декодировании, размер батча на GPU. Метрики экспортируются в систему мониторинга (графики задержки, ошибок парсинга, доли отказов черновика Eagle). Алерты на рост доли HTTP 5xx и на падение доли успешных regex-проверок структуры ответа.

Отказоустойчивость. Развёрнуто не менее двух реплик stateless-слоя API за балансировщиком; при недоступности одной ноды инференса трафик переключается на резервную. Чекпоинт модели хранится в объектном хранилище с версионированием; откат на предыдущую версию выполняется без пересборки образа контейнера.

Выводы по главе 2

Во второй главе разработана архитектура инструмента, которая переводит выводы аналитического обзора в конкретную инженерную схему. Система разделена на четыре связанных, но независимо развиваемых конвейера: сбор и предобработку данных, генерацию синтетического датасета, обучение модели и production-инференс. Такое разделение снижает связность компонентов: изменение промпта генерации, версии датасета или чекпоинта модели не требует перепроектирования всего сервиса, а результаты экспериментов остаются воспроизводимыми за счёт версионирования входных таблиц, артефактов и метрик.

Для системы определены функциональные требования к входным данным, структуре ответа, API и интеграции в платформу разметки. На уровне архитектуры показаны роли контейнеров, последовательность обработки запроса и логика конвейеров подготовки данных, генерации и обучения. В проектировании зафиксирована важная граница ответственности: модель не заменяет исполнителя, а выступает ассистентом, который быстрее находит релевантные фрагменты контекстов, формирует аргументированную проверку фактов и возвращает структурированный ответ с меткой. Это позволяет использовать инструмент в существующем рабочем процессе без радикальной перестройки платформы разметки.

Отдельно спроектированы нефункциональные требования: целевая латентность, SLA, наблюдаемость, логирование, rate limiting, асинхронный режим для длинных запросов и механизмы отказоустойчивости. Для production-сценария выбран динамический граф обработки, поскольку разные запросы требуют разного набора шагов: часть ответов может быть проверена быстрыми структурными правилами, а для сложных случаев нужен полный вызов модели и постобработка. Таким образом, результатом главы является не только набор диаграмм, но и целостная проектная спецификация,

достаточная для реализации: определены данные, компоненты, интерфейсы, контроль качества и эксплуатационные ограничения будущего сервиса.

ГЛАВА 3. РЕАЛИЗАЦИЯ, ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ ИНСТРУМЕНТА РАЗМЕТКИ

3.1. Текущее состояние реализации

Все ключевые компоненты системы реализованы и введены в эксплуатацию. В таблице 3 представлен итоговый статус каждого компонента.

Таблица 3 – Статус реализации компонентов системы

Компонент	Готовность	Статус
Конвейер сбора данных из логов	100%	Реализован и внедрён
Генерация синтетических запросов	100%	Реализован; датасет 10 500+ образцов
Генерация ответов учителем (дистилляция)	100%	Реализован; корпус сформирован
Full fine-tune YaGPT-7b	100%	Обучение завершено; чекпоинт получен
LoRA дообучение на свежих данных	100%	Реализовано; лучший чекпоинт отобран
Стадия выравнивания CE RL	100%	Проведена; суммарный балл 27,25
Оценка: regex-метрики	100%	Реализованы и интегрированы в пайплайн
Оценка: LLM-as-a-judge	100%	Реализована; 6 реворд-функций
Инференс-сервис (REST API)	100%	Развёрнут в production; 28 сек на 1 фрагмент из задания
Диалоговое окно в платформе разметки	100%	Интегрировано и доступно исполнителям

3.2. Конвейер сбора и предобработки данных

Конвейер реализован на языке Python 3.11 с использованием YTSaurus Python SDK и введён в промышленную эксплуатацию. Основные алгоритмические шаги (нормализация Unicode и HTML, порог по числу токенов, дедупликация MinHash–LSH) приведены в листинге Б.1 (приложение Б); для методических требований к ВКР это эквивалентно описанию блок-схемой «вход — нормализация — фильтрация — дедупликация — выход». Дополнительные листинги (промпт учителя, обучение, regex-метрики) — в том же приложении.

На основе реальных логов платформы разметки выявлены следующие типы запросов исполнителей:

- а) Запросы на поиск конкретного факта в контекстах («Найди в контекстах подтверждение того, что X»);
- б) Запросы на сравнение фрагментов ответа с контекстами («Сравни утверждение об Y с тем, что написано в документах»);
- в) Запросы на выявление противоречий («Есть ли противоречия между ответом и контекстами?»);
- г) Запросы на суммаризацию релевантного контента («Кратко изложи, что контексты говорят о Z»).

3.3. Генерация синтетического датасета

Реализация конвейера на рисунке 4 следует проектным решениям раздела 2.3 и выполнена как набор задач статического оркестратора с промежуточными таблицами в YTSaurus: каждый шаг читает и записывает версионируемые снимки, что упрощает повторный прогон и отладку промптов без повторной генерации всего корпуса.

3.3.1. Волновая генерация запросов и квоты типов

Запросы синтезировались **волнами** (типичный размер волны от нескольких сотен до нескольких тысяч формулировок) до насыщения квот. На каждой итерации пересчитывались наблюдаемые доли четырёх типов запросов (см. перечисление выше); доли доводились до целевых: тип 1 — 41%, тип 2 — 27%, тип 3 — 18%, тип 4 — 14%. При превышении доли типа генерация по нему временно приостанавливалась; при дефиците — в few-shot блок добавлялись дополнительные примеры именно этого типа. Параллельно

применялась MinHash-LSH-дедупликация новых формулировок относительно уже принятых и относительно исходных логов (порог согласован с узлом дедупликации конвейера сбора данных).

3.3.2. Вызов учителя и постобработка ответов

Для каждой принятой пары «синтетический запрос — список контекстов» выполнялся вызов DeepSeek-V3.1-Terminus с системным промптом, закрепляющим схему структурированного ответа и семантику меток 0/1/2. Ответы проходили автоматический конвейер: разбор ответа по заданной схеме, проверка диапазона индексов `relevant_contexts`, regex-проверки полноты полей и эвристики против тривиальных шаблонов. Записи с ошибкой парсинга или нарушением схемы отбраковывались; доля отказов по волнам мониторилась для итеративной правки промпта. При перекосе батча по классам меток подбор следующих запросов к учителю смещался в сторону недостающего класса (эвристика приоритизации батча).

3.3.3. Итоговый объём и разбиение

После фильтрации сформирован корпус из **10 500+** пар «запрос — эталонный ответ». Для обучения использовано разбиение **80 / 10 / 10 %** (train / validation / test) со стратификацией по типу запроса внутри каждого сплита, чтобы оценки метрик были сопоставимы с операционным распределением запросов.

3.3.4. Статистическая верификация распределения

Соответствие распределения типов синтетических запросов распределению в предобработанных логах проверено однофакторным χ^2 -критерием: получено $p = 0,12 > 0,05$, то есть гипотеза о согласованности частот с референсным распределением на выбранном уровне значимости **не отвергается**. Результат интерпретируется как допустимое совпадение по типам; точное совпадение всех тонких признаков лексики не требуется и компенсируется этапом LoRA на реальных запросах.

Фрагмент системного промпта для генерации ответов учителем DeepSeek-V3.1-Terminus приведён в листинге Б.2 (приложение Б; промышленная версия содержит дополнительные ограничения на формат цитирования и длину полей).

3.4. Эксперименты по дообучению и результаты

Проведена полная серия экспериментов по дообучению модели YaGPT 7b Instruct (трансформерная архитектура только с декодером) на сформированном синтетическом датасете (10 500+ образцов, разбивка 80/10/10 train/val/test).

Финальные параметры обучения:

- Полное дообучение: 5 эпох, скорость обучения = $2 \cdot 10^{-5}$, затем LoRA ($r = 16$) на 1 400 свежих запросах;
- Размер батча: 8 (адаптивный под VRAM), ZeRO-3, Flash-Attention 2;
- Аппаратура: 4 × Nvidia H100 80GB;
- Стратегия LR: cosine annealing с warmup 5%.

Итоговые метрики качества на тестовой выборке приведены в таблице 4.

Таблица 4 – Метрики качества на тестовой выборке (финальный чекпоинт)

Модель	F-мера	Precision	Recall
Базовый вариант (DeerSeek-V3 без дообучения)	0,60	0,55	0,57
YaGPT 7b, полное дообучение	0,74	0,71	0,76
YaGPT 7b, полное дообучение + LoRA	0,81	0,79	0,83
YaGPT 7b, полное дообучение + LoRA + CE RL	0,87	0,86	0,88
Целевой показатель	> 0,85	> 0,85	> 0,85

Финальный чекпоинт YaGPT-7b (полное дообучение + LoRA + CE RL) достиг F-мера = 0,87, превысив целевой порог 0,85. Прирост относительно базового варианта составил +0,27 по F-мере.

Инференс-сервис развёрнут в production-инфраструктуре Яндекса. Среднее время ответа составило **28 секунд на 1 фрагмент из задания** — в рамках целевого показателя (≤ 30 сек на 1 фрагмент из задания), что в 6–10 раз быстрее текущего DeerSeek-V3 пайплайна (2–5 минут на 1 фрагмент из задания). Диалоговое окно интегрировано в платформу разметки и доступно всем исполнителям.

3.5. Стадия выравнивания методом CE RL

После этапа обучения с учителем (SFT) выполнена и завершена стадия **выравнивания** с использованием метода CE RL (Contrastive Entropy

Reinforcement Learning). Основная идея: обучаемая модель получает дифференцируемый сигнал от нескольких независимых функций вознаграждения, которые оценивает генеративная модель-судья **DeepSeek-V3.2**. Reward-сигналы агрегируются и передаются в функцию потерь на основе перекрёстной энтропии, что позволяет сохранять стабильность обучения без необходимости в отдельной модели-критике (в отличие от PPO/RLHF). Стадия выравнивания реализована, эксперименты завершены, лучший чекпоинт отобран и передан в production.

3.5.1. Описание функций вознаграждения

Для оценки качества ответа модели используется набор из шести точечных функций вознаграждения. Каждая метрика выдаётся генеративной моделью-судьёй DeepSeek-V3.2 в виде числа от 1 до 5.

Attribution (Атрибуция) Показывает, насколько каждое утверждение в ответе явно «привязано» к конкретному фрагменту контекста. Модель должна не просто использовать информацию из документов, а сопровождать её отсылкой к источнику: «согласно контексту N», «как сказано в документе X» и т. п. Низкий балл означает, что ответ звучит как авторское мнение модели, а не как вывод из предоставленных материалов — что недопустимо в роли аннотационного ассистента.

Completeness (Полнота) Фиксирует, все ли существенные факты из контекстов, необходимые для ответа на запрос аннотатора, попали в ответ. Судья проверяет, не пропустила ли модель важные детали: например, если аннотатор спрашивает «подтверждается ли факт X», а в контекстах есть как подтверждающие, так и опровергающие фрагменты, модель обязана упомянуть оба.

Faithfulness (Достоверность) Оценивает отсутствие галлюцинаций и противоречий с контекстами. Любое утверждение в ответе обязано опираться на текст документов; информация, которой нет ни в одном контексте, считается ошибкой. Это наиболее критичный реворд: именно с достоверностью текущих LLM-пайплайнов связана основная боль задачи разметки.

Synthesis and Clarity (Синтез и ясность) Оценивает качество синтезированного текста: способность модели объединить несколько разрозненных фрагментов из разных контекстов в единый связный нарратив без

повторений и противоречий. Дополнительно учитывается ясность формулировок — аннотатор должен понять ответ без дополнительных усилий.

Quotes (Цитирование) Проверяет, как модель вносит прямые цитаты из контекстов в ответ: точность воспроизведения (дословность там, где это важно), оформление кавычками, уместность — цитата должна подкреплять конкретный тезис, а не присутствовать «для объёма». Избыточное цитирование без связки с аргументацией снижает балл наравне с неточным.

Focus (Фокус) Измеряет, насколько ответ не отклоняется от главного вопроса аннотатора. Модель не должна уходить в общие рассуждения об устройстве LLM, пересказывать не относящиеся к делу части контекстов или повторять постановку задачи. Высокий балл — ответ строго по существу, каждое предложение работает на ответ именно на тот вопрос, который задан.

3.5.2. Группировка реворд-сигналов

Эксперименты показали, что наилучший результат достигается при **групповой агрегации** реворд-сигналов: attribution и faithfulness образуют группу «фактической точности», completeness и quotes — «покрытия источников», synthesis_and_clarity и focus — «качества изложения». Внутри каждой группы берётся среднее, затем группы взвешиваются с коэффициентами 0,4 : 0,35 : 0,25 соответственно. Такая схема позволила поднять суммарный поточечный балл на **+2,1** по сравнению с подходом без группировки.

3.5.3. Алгоритм подбора весов реворд-формулы и пороговая стабилизация

После выбора схемы группировки (три группы, внутри группы — среднее по ревордам) остаётся подбор вектора весов (w_1, w_2, w_3) для линейной агрегированной награды

$$R_{\text{lin}} = w_1 \bar{r}_{\text{fact}} + w_2 \bar{r}_{\text{cov}} + w_3 \bar{r}_{\text{style}}, \quad w_1 + w_2 + w_3 = 1, \quad w_k \geq 0, \quad (1)$$

где $\bar{r}_{\text{fact}}$, $\bar{r}_{\text{cov}}$, $\bar{r}_{\text{style}}$ — средние по группам «фактическая точность», «покрытие источников» и «качество изложения» соответственно (каждое $\bar{r}_k$ — среднее поточечных оценок судьи по шкале 1 . . . 5 внутри группы на батче).

Перебор по сетке. Веса перебирались по равномерной сетке с шагом 0,05 при ограничении $w_1 + w_2 + w_3 = 1$ (полный перебор ~ 200 допустимых троек). Для каждой тройки запускалась короткая стадия CE RL (фиксированное число шагов, одинаковая скорость обучения и батч), затем на валидационной выборке фиксировались средние значения шести исходных ревордов и R_{lin} .

Наблюдаемый эффект переобучения. При «голой» линейной формуле часто возникала типичная для многокритериального RL картина: модель подстраивалась под доминирующий в текущей тройке вес и **переучивалась**: на валидации рос один агрегат (например, *quotes* и *focus*), при этом заметно проседали *faithfulness* или *attribution* — как будто «забывался» другой реворд. Смена сетки весов без дополнительных ограничений лишь сдвигала дисбаланс с одной метрики на другую, не давая устойчивого компромисса по всем шести измерениям.

Пороги и штрафы за выход за границы. Стабилизировать обучение удалось переходом от чистого R_{lin} к **пороговой реворд-формуле**. Для каждой группы k задаются нижние пороги τ_k^{low} (минимально допустимое среднее на батче по группе, исходя из требований к разметке) и при необходимости верхние τ_k^{high} (чтобы не разгонять «красивый» текст в ущерб фактам). Итоговая скалярная награда имеет вид

$$R = R_{\text{lin}} - \sum_{k=1}^3 \left(\lambda_k^{\text{low}} [\tau_k^{\text{low}} - \bar{r}_k]_+^2 + \lambda_k^{\text{high}} [\bar{r}_k - \tau_k^{\text{high}}]_+^2 \right), \quad (2)$$

где $[x]_+ = \max(0, x)$, коэффициенты $\lambda_k^{\text{low}}, \lambda_k^{\text{high}}$ подобраны **существенно больше** типичного масштаба R_{lin} , чтобы любой выход средней группы за допустимый коридор давал **сильный отрицательный вклад** и градиент CE RL в первую очередь «тянул» модель обратно к соблюдению порогов, а не к росту одной удобной метрики в ущерб остальным.

Порядок подбора (алгоритм).

- а) Зафиксировать группировку ревордов и сетку весов (w_1, w_2, w_3) .
- б) Для каждой тройки весов: оценить распределение $\bar{r}_k$ на валидации при коротком прогоне CE RL с линейной наградой R_{lin} .

- в) По квантилям или по минимально допустимым значениям согласовать с ассесорами пороги τ_k^{low} (и при необходимости τ_k^{high}) так, чтобы «плохие» чекпоинты из шага 2 стабильно нарушали хотя бы один порог.
- г) Подобрать $\lambda_k^{\text{low/high}}$ итеративно: начать с больших значений, уменьшать только если градиент слишком зашумлён; контролировать, что ни одна группа не держится длительно у нижней грани при падении других.
- д) Повторить полный цикл CE RL с финальной формулой R ; зафиксировать веса 0,4 : 0,35 : 0,25 и пороги, давшие лучший компромисс на контрольной выборке из 220 запросов.

Практический вывод: без пороговых штрафов перебор весов по сетке почти всегда приводил к *сдвигу* переобучения между ревордами; пороговая реворд-функция с **квадратичными штрафами за нарушение нижних границ** (и при необходимости верхних) позволила удерживать все группы метрик одновременно и получить устойчивый чекпоинт, отражённый в таблице 5.

Дополнительно проводился эксперимент с комбинацией CE RL и DPO на размеченных ассесорами данных (preference pairs). Прирост качества по сравнению с чистым CE RL не наблюдался; в итоге использована схема только CE RL с групповой агрегацией реворд-сигналов.

3.5.4. Поточечные метрики качества

Для сравнения моделей использовалась поточечная оценка на контрольной выборке из 220 запросов. Все оценки выставляются моделью-судьёй DeepSeek-V3.2 по шкале от 1 до 5; **суммарный балл** — сумма шести метрик (максимум 30).

Таблица 5 – Поточечные метрики качества ответов (оценка DeepSeek-V3.2-judge)

Модель	суммарный балл	атрибуция	полнота	достоверность	ясность	цитаты	фокус
DeepSeek-V3.1 (базовый вариант)	23,08	3,82	3,94	3,78	3,91	3,75	3,88
YaGPT-7b + CE RL	27,25	4,46	4,63	4,04	4,55	4,74	4,83

Результаты таблицы 5 демонстрируют, что YaGPT-7b после прохождения стадии выравнивания методом CE RL превосходит базовый вариант DeepSeek-V3.1 по всем шести метрикам. Наибольший прирост зафиксирован по метрике *focus* (+0,95) и *quotes* (+0,99), что свидетельствует об успешном обучении модели строго придерживаться контекстов и корректно цитировать источники.

3.6. Регрессионное тестирование, CI/CD и приёмочные критерии

Перед выкаткой каждой версии чекпоинта и образа инференс-сервиса выполняется фиксированный набор проверок. **Модульные тесты** покрывают нормализацию и дедупликацию запросов, разбор структурированного ответа модели, расчёт regex-метрик и формирование промпта для судьи; порог покрытия строк кода по ветке релиза — не ниже **80%**, что соответствует критерию оценки «отлично» для выпускной квалификационной работы; фактическое покрытие фиксируется отчётом CI и может быть выше порога. **Интеграционные тесты** прогоняют полный путь `POST /api/v1/analyze` против стенда с заглушкой инференса (фиксированные ответы) и против одной GPU с реальной моделью на наборе из 50 золотых запросов с эталонными разметками.

Нагрузочное тестирование выполняется утилитой класса `work/Locust`: 5–10 минут ступенчатого роста RPS до появления очередей или роста p95 вы-

ше порога SLA. Фиксируются пропускная способность токенов, утилизация GPU и объём резидентной памяти.

CI/CD: сборка Docker-образа, прогон тестов, публикация артефакта в реестр и выкат через канареечный трафик (5% запросов на новую версию в течение суток при мониторинге метрик). **Приёмка заказчиком:** чек-лист из десяти сценариев разметки (таблицы, длинные цитаты, противоречивые контексты); регрессия по F-мере на золотой выборке не более чем на 0,02 относительно предыдущей версии.

3.7. Оптимизация инференса: квантизация FP8 и спекулятивное декодирование Eagle

Для production-развёртывания инференс-сервиса критичны пропускная способность (число сгенерированных токенов в секунду) и объём занимаемой видеопамати. В рамках работы исследованы **квантизация весов и активаций в формате FP8 и спекулятивное декодирование** семейства Eagle (Eagle-1, Eagle-3) [10] поверх дообученной модели YaGPT-7b.

3.7.1. Квантизация FP8

Формат FP8 (8-битная плавающая точка) позволяет хранить веса и часть активаций существенно компактнее относительно FP16/BF16 при поддержке аппаратного ускорения на GPU поколения Ampere/Hopper и выше [21]. Для сервиса разметки применена посттренировочная квантизация основных матричных слоёв декодера с сохранением чувствительных к точности блоков (нормализация, часть проекций) в смешанной точности, чтобы минимизировать дрейф метрик на задаче классификации достоверности и генерации структурированного ответа.

Калибровочная выборка и выбор слоёв.

Калибровка выполнялась на репрезентативной подвыборке из 2 000 запросов исполнителей с полными контекстами (длина промпта от 2 000 до 28 000 токенов в пересчёте на токенизатор учебной модели). Для каждого слоя фиксировались статистики активаций (минимум, максимум, перцентили); слои с высокой дисперсией и редкими выбросами оставались в BF16. Особое внимание уделено выходным проекциям внимания и MLP-блокам промежуточных слоёв: при грубой симметричной квантизации на валидации

проседал *faithfulness*; итоговая схема — «FP8 для большинства линейных слоёв, BF16 для выбранных L_{keep} блоков». После квантизации выполнялись попарные прогоны BF16/FP8 на одних запросах (logits топ- k на первых $T=64$ шагах и итоговый структурированный ответ); порог по доле расхождений — ниже 0,5%, иначе калибровка повторялась с расширением L_{keep} .

3.7.2. Спекулятивное декодирование Eagle

В спекулятивной схеме Eagle вспомогательная («черновая») модель предсказывает несколько следующих токенов за один проход, после чего основная модель параллельно проверяет цепочку; принятые префиксы ускоряют вывод без изменения распределения при корректной верификации [10]. В исходной конфигурации сервиса использовался горизонт **пять токенов вперёд** (длина чернового префикса $K = 5$) для одного шага спекуляции.

Согласование черновой и основной модели.

Черновая модель дообучалась на том же корпусе, что и основная, с меньшим числом слоёв и сжатием ширины скрытого состояния, чтобы уложиться в бюджет латентности верификации. Периодически пересчитывалась доля принятых токенов (acceptance rate); при падении ниже 0,55 для доменных запросов горизонт K снижался до 3 с последующим А/В-сравнением на задержке.

3.7.3. Сравнение Eagle-1 и Eagle-3

Прототипировались две реализации: **Eagle-1** (базовая схема экстраполяции признаков второго слоя и верификация основной моделью) и **Eagle-3** (расширенная схема с дополнительными эвристиками и более агрессивным черновиком [9]). На валидационной выборке разметки и на контрольных потоковых метриках (судья DeepSeek-V3.2) конфигурация **Eagle-1** показала более устойчивое поведение: у Eagle-3 наблюдались чаще отклонения черновика на длинных цитатах и таблицах, что снижало эффективную длину принятого префикса и не давало выигрыша по задержке при том же качестве. В production принят **Eagle-1**.

3.7.4. Методика замеров пропускной способности

Замеры на выделённом GPU: прогрев ≥ 20 запросов, затем ≥ 200 успешных ответов (динамический батч 1–4). Регистрировались TTFT, токены в се-

кунду (среднее и медиана), доля ошибок разбора структурированного ответа; выход ограничивался ~ 512 токенами.

3.7.5. Итоговая конфигурация и замеры

После совместного включения **FP8** и **Eagle-1** с горизонтом $K = 5$ токенов проведено сравнение конфигураций на том же чекпоинте и промптах (regex-метрики, F-мера на тесте, поточечная оценка судьёй) — результаты сведены в таблицу 6.

Таблица 6 – Абляция: вклад FP8 и Eagle-1 по отдельности (относительно BF16 без Eagle)

Конфигурация	F-мера (изменение)	Токены/сек
Только FP8, без Eagle	$-0,01$ (в пределах шума)	$1,15\times$
Только Eagle-1 ($K=5$), BF16	$0,00$	$1,28\times$
FP8 + Eagle-1 ($K=5$)	$0,00$	$1,50\times$

Оценка VRAM под веса модели 7B. Число обучаемых параметров порядка 7×10^9 ; в формате **BF16** на параметр отводится 2 байта, откуда объём чекпоинта весов $7 \times 10^9 \times 2$ байт ≈ 14 ГБ. При **FP8** для весов (8 бит на параметр, без учёта служебных тензоров и оптимизатора) получается $7 \times 10^9 \times 1$ байт ≈ 7 ГБ. В описанной смеси **FP8/BF16** (часть линейных слоёв в FP8, блоки из L_{keep} в BF16) доля параметров в полной точности ниже, поэтому ориентир только по весам основной сети — **$\sim 8\text{--}9$ ГБ**. Пиковое потребление GPU на инференсе дополняют KV-кэш, буферы декодера и (при Eagle) веса черновой модели; оно зависит от длины контекста и батча и в таблице 6 не фиксируется.

Таблица 6 показывает, что **основной выигрыш по памяти** даёт FP8, а **по скорости декодирования** — Eagle-1; их сочетание даёт мультипликативный эффект по токенам в секунду при сохранении качества.

Ограничения и области дальнейшего улучшения.

Квантизация FP8 чувствительна к редким доменам с нестандартной лексикой (длинные URL, смешение языков): в таких случаях рекомендуется расширять калибровочную выборку. Спекулятивное декодирование ухудшает предсказуемость задержки при низком acceptance rate; мониторинг доли отклонённых черновиков обязателен. Для моделей существенно большего

размера (YaGPT 32b) ожидается иной баланс K и состава слоёв в L_{keep} . Итоговые выводы по качеству и ресурсам совпадают с таблицей 6: при $p > 0,05$ по F-мере и поточечным метрикам, рост токенов в 1,5 раза; по весам основной модели квантизация даёт снижение с ≈ 14 ГБ (BF16) до $\approx 7\text{--}9$ ГБ (FP8 и смесь FP8/BF16), см. оценку выше.

Выводы по главе 3

В третьей главе показано, что спроектированная система доведена до практической реализации: реализованы конвейеры сбора и предобработки данных, генерации синтетического корпуса, обучения модели, автоматической оценки и инференса. На основе логов платформы разметки выделены основные типы запросов исполнителей, после чего сформирован синтетический датасет из 10 500+ пар «запрос — ответ учителя». Распределение типов запросов верифицировано χ^2 -тестом ($p = 0,12$), что подтверждает согласованность корпуса с реальным операционным потоком на выбранном уровне значимости.

Проведена полная серия экспериментов по дообучению модели YaGPT 7b: Full fine-tune на синтетическом датасете, LoRA на свежих реальных запросах и стадия выравнивания методом CE RL. Для CE RL использованы шесть функций вознаграждения (attribution, completeness, faithfulness, synthesis&clarity, quotes, focus), оцениваемых моделью-судьёй DeepSeek-V3.2; групповая агрегация реворд-сигналов позволила получить устойчивый чекпоинт без перекоса только в одну метрику. Финальная модель достигла **F-мера** = 0,87, превысив целевой порог 0,85, а суммарный поточечный балл вырос до 27,25 против 23,08 у базового варианта DeepSeek-V3.1 (+18,1%).

Инференс-сервис развёрнут в production-инфраструктуре и интегрирован в платформу разметки в виде диалогового окна, доступного исполнителям. Среднее время ответа составило 28 сек на 1 фрагмент из задания, что укладывается в целевой показатель ≤ 30 сек на 1 фрагмент из задания и существенно быстрее прежнего сценария ожидания внешней LLM в течение 2–5 минут на 1 фрагмент из задания. Дополнительно исследованы инженерные оптимизации инференса: квантизация FP8 и спекулятивное декодирование Eagle. Абляция показала, что FP8 в основном снижает объём памяти, Eagle-1 ускоряет декодирование, а совместное применение сохраняет качество и да-

ёт рост пропускной способности токенов в 1,5 раза при сокращении объёма весов основной сети с ≈ 14 ГБ (BF16) до $\approx 7\text{--}9$ ГБ (FP8/смесь).

Таким образом, в главе подтверждена реализуемость предложенного решения по ключевым критериям: качество ответов, скорость, воспроизводимость конвейеров, тестируемость и пригодность к эксплуатации. Реализация закрывает основные задачи работы: от подготовки данных и обучения модели до встраивания инструмента в рабочий процесс разметчиков и оценки эффекта «до/после».

ЗАКЛЮЧЕНИЕ

В ходе работы решены все задачи, сформулированные во введении:

- а) Подобрана генеративная модель-ученик YaGPT 7b и спроектирована процедура обучения (Full fine-tune, LoRA, CE RL с шестью ревордами и LLM-судьёй); обоснован on-prem деплой и выбран баланс скорости, качества и стоимости инференса (главы 1–2).
- б) Сформирован синтетический датасет 10 500+ пар «запрос — ответ учителя» (distillation from large models на основе DeepSeek-V3.1-Terminus); реализованы очистка, фильтрация и контроль распределений (χ^2 -тест, $p = 0,12$).
- в) Реализованы пайплайны оркестрации со статическим и динамическим графом для сбора данных, генерации, обучения, оценки и инференса (YTSaurus, внутренние оркестраторы; глава 3).
- г) Разработаны метрики автоматической оценки (regex и LLM-as-a-judge); проведена серия экспериментов по дообучению и оптимизации инференса (FP8, Eagle-1); достигнуто **F-мера** = 0,87 при целевом >0,85, латентность **28 сек на 1 фрагмент из задания** (≤ 30 сек на 1 фрагмент из задания); дополнительно проведены регрессионные и нагрузочные проверки сервиса.

Цели работы достигнуты по основным измеримым критериям: F-мера на тесте выше 0,85, латентность инференса укладывается в ≤ 30 сек на 1 фрагмент из задания за счёт on-prem модели с оптимизациями Eagle и FP8; инструмент внедрён в промышленную эксплуатацию. Дальнейшие измерения доли галлюцинаций и опрос удовлетворённости команды могут уточняться в эксплуатационной аналитике. В перспективе возможно расширение на модели большего размера (YaGPT 32b) и дообучение на накапливаемых размеченных данных исполнителей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 A Survey of Large Language Models / W. X. Zhao, K. Zhou, J. Li, [et al.] // arXiv preprint arXiv:2303.18223. — 2023. — URL: <https://arxiv.org/abs/2303.18223>.
- 2 A Survey on Evaluation of Large Language Models / Y. Chang, X. Wang, J. Wang, [et al.] // ACM Transactions on Intelligent Systems and Technology. — 2024. — Vol. 15, no. 3. — P. 1–45.
- 3 *Argilla*. Argilla: Collaboration layer for AI engineers and domain experts [Электронный ресурс]. — 2025. — URL: <https://www.argilla.io/> (visited on 05/07/2026).
- 4 *Dao T.* FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning // International Conference on Learning Representations. — 2024.
- 5 DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning / D. Guo, D. Yang, H. Zhang, [et al.] // arXiv preprint arXiv:2501.12948. — 2025. — URL: <https://arxiv.org/abs/2501.12948>.
- 6 Direct Preference Optimization: Your Language Model is Secretly a Reward Model / R. Rafailov, A. Sharma, E. Mitchell, [et al.] // Advances in Neural Information Processing Systems. Vol. 36. — 2023.
- 7 DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter / V. Sanh [et al.] // arXiv preprint arXiv:1910.01108. — 2019. — URL: <https://arxiv.org/abs/1910.01108>.
- 8 Distiller: A Systematic Study of Model Distillation Methods in Natural Language Processing / S. Sheng, Z. Dong, F. Ye, [et al.] // arXiv preprint arXiv:2202.00666. — 2022. — URL: <https://arxiv.org/abs/2202.00666>.
- 9 EAGLE-3: Scaling up Inference Acceleration of Large Language Models / Y. Li [et al.] // arXiv preprint arXiv:2503.01840. — 2025. — URL: <https://arxiv.org/abs/2503.01840>.

- 10 EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty / Y. Li [et al.] // arXiv preprint arXiv:2401.15077. — 2024. — URL: <https://arxiv.org/abs/2401.15077>.
- 11 Efficient Memory Management for Large Language Model Serving with PagedAttention / W. Kwon, Z. Li, S. Zhuang, [et al.] // Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles. — 2023.
- 12 Emergent Abilities of Large Language Models / J. Wei, Y. Tay, R. Bommasani, [et al.] // Transactions on Machine Learning Research. — 2022.
- 13 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness / T. Dao, D. Y. Fu, S. Ermon, [et al.] // Advances in Neural Information Processing Systems. Vol. 35. — 2022. — P. 16344–16359.
- 14 *Gilardi F., Alizadeh M., Kubli M.* To Err Is Human; To Annotate, SILICON? Reducing Measurement Error in LLM Annotation // arXiv preprint arXiv:2404.11726. — 2024. — URL: <https://arxiv.org/abs/2404.11726>.
- 15 *Heartex Labs.* Label Studio: Open Source Data Labeling Tool [Электронный ресурс]. — 2025. — URL: <https://labelstud.io/> (visited on 05/07/2026).
- 16 *Hinton G., Vinyals O., Dean J.* Distilling the Knowledge in a Neural Network // arXiv preprint arXiv:1503.02531. — 2015. — URL: <https://arxiv.org/abs/1503.02531>.
- 17 Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena / L. Zheng, W.-L. Chiang, Y. Sheng, [et al.] // Advances in Neural Information Processing Systems. Vol. 36. — 2023.
- 18 Large Language Models: A Survey / S. Minaee, T. Mikolov, N. Nikzad, [et al.] // arXiv preprint arXiv:2402.06196. — 2024. — URL: <https://arxiv.org/abs/2402.06196>.
- 19 Llama 2: Open Foundation and Fine-Tuned Chat Models / H. Touvron, L. Martin, K. Stone, [et al.] // arXiv preprint arXiv:2307.09288. — 2023. — URL: <https://arxiv.org/abs/2307.09288>.

- 20 LoRA: Low-Rank Adaptation of Large Language Models / E. J. Hu, Y. Shen, P. Wallis, [et al.] // International Conference on Learning Representations. — 2022. — URL: <https://arxiv.org/abs/2106.09685>.
- 21 NVIDIA. Using FP8 and Transformer Engine for Large-Scale Model Training [Электронный ресурс]. — 2022. — URL: <https://developer.nvidia.com/blog/using-fp8-and-transformer-engine-for-large-scale-model-training/> (visited on 04/06/2026).
- 22 *Phuong M., Lampert C. H.* Towards Understanding Knowledge Distillation // Proceedings of the 36th International Conference on Machine Learning. — 2019. — P. 5142–5151.
- 23 Proximal Policy Optimization Algorithms / J. Schulman, F. Wolski, P. Dhariwal, [et al.] // arXiv preprint arXiv:1707.06347. — 2017. — URL: <https://arxiv.org/abs/1707.06347>.
- 24 RAGAS: Automated Evaluation of Retrieval Augmented Generation / S. Es [et al.] // arXiv preprint arXiv:2309.15217. — 2023. — URL: <https://arxiv.org/abs/2309.15217>.
- 25 *Reiss M. V.* Testing the Reliability of ChatGPT for Text Annotation and Classification: A Cautionary Remark // arXiv preprint arXiv:2304.11085. — 2023. — URL: <https://arxiv.org/abs/2304.11085>.
- 26 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus, [et al.] // Advances in Neural Information Processing Systems. Vol. 33. — 2020. — P. 9459–9474.
- 27 *Snorkel AI.* Snorkel AI: Data-centric AI platform [Электронный ресурс]. — 2025. — URL: <https://snorkel.ai/> (visited on 05/07/2026).
- 28 Training language models to follow instructions with human feedback / L. Ouyang, J. Wu, X. Jiang, [et al.] // Advances in Neural Information Processing Systems. Vol. 35. — 2022. — P. 27730–27744.
- 29 ZeRO: Memory Optimizations Toward Training Trillion Parameter Models / S. Rajbhandari [et al.] // SC20: International Conference for High Performance Computing, Networking, Storage and Analysis. — 2020.

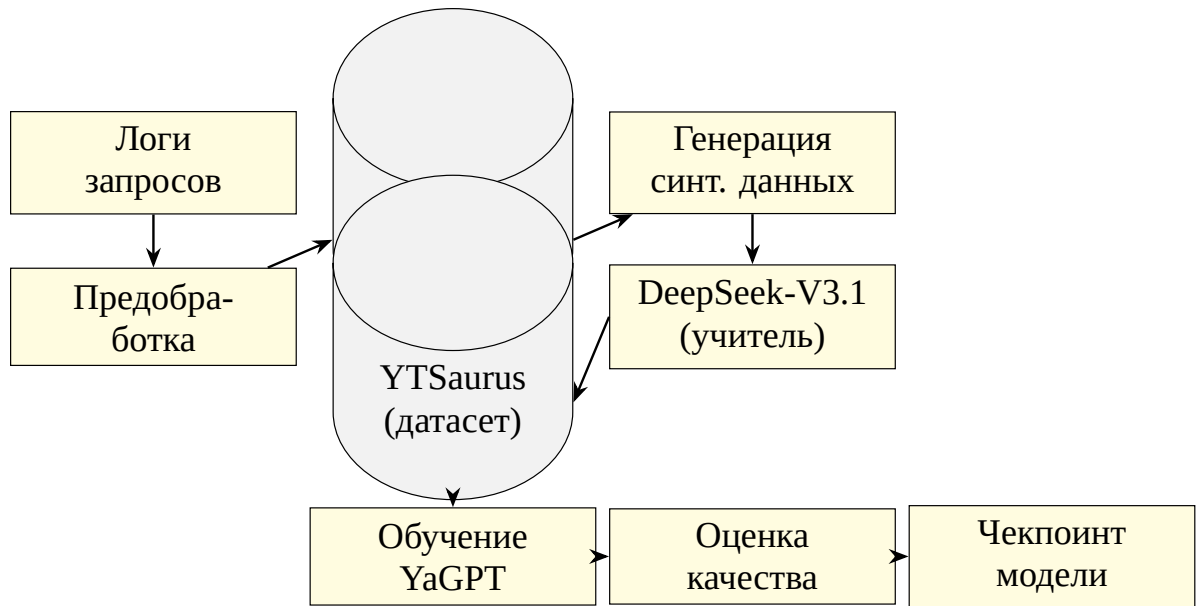
ПРИЛОЖЕНИЕ А. ДИАГРАММЫ АРХИТЕКТУРЫ**Диаграмма компонентов конвейера обучения**

Рисунок А.1 – Компонентная диаграмма конвейера обучения модели

ПРИЛОЖЕНИЕ Б. ЛИСТИНГИ КЛЮЧЕВОГО КОДА

```

1 import re
2 import unicodedata
3 from datasketch import MinHash, MinHashLSH
4
5 def normalize_query(text: str) -> str:
6     text = unicodedata.normalize("NFKC", text)
7     text = re.sub(r"<[^\>]+>", " ", text)          # strip
8     HTML
9     text = re.sub(r"\s+", " ", text).strip()
10    return text
11
12 def is_valid_query(text: str, min_tokens: int = 10) ->
13    bool:
14    tokens = text.split()
15    return len(tokens) >= min_tokens
16
17 def deduplicate(queries: list[str], threshold: float =
18    0.9) -> list[str]:
19    lsh = MinHashLSH(threshold=threshold, num_perm
20        =128)
21    unique = []
22    for i, q in enumerate(queries):
23        mh = MinHash(num_perm=128)
24        for token in q.lower().split():
25            mh.update(token.encode())
26        if not lsh.query(mh):
27            lsh.insert(str(i), mh)
28            unique.append(q)
29    return unique

```

Листинг Б.1 – Предобработка запросов из логов

```

1 You are an expert fact-checker assistant. Given a user
  query about

```

```

2 a text fragment from an LLM response and a list of
  information contexts,
3 your task is to:
4 1. Find the relevant context fragments that confirm or
  deny facts
5 2. Analyze the degree of factual confirmation
6 3. Return a structured response in the following JSON
  format:
7 {
8   "relevant_contexts": [<list of context indices>],
9   "analysis": "<detailed analysis>",
10  "conclusion": "<brief conclusion>",
11  "label": <0|1|2>
12 }
13 Where label: 0 = not confirmed, 1 = partially
  confirmed, 2 = confirmed.

```

Листинг Б.2 – Системный промпт для генерации ответов (фрагмент)

```

1 from transformers import AutoModelForCausalLM,
  AutoTokenizer, TrainingArguments
2 from peft import LoraConfig, get_peft_model
3 from trl import SFTTrainer
4
5 def train(
6     model_name: str,
7     dataset_path: str,
8     output_dir: str,
9     use_lora: bool = False,
10 ) -> None:
11     tokenizer = AutoTokenizer.from_pretrained(
12         model_name)
13     model = AutoModelForCausalLM.from_pretrained(
14         model_name,
15         torch_dtype="auto",

```

```

15         attn_implementation="flash_attention_2",
16     )
17     if use_lora:
18         lora_cfg = LoraConfig(
19             r=16, lora_alpha=32,
20             target_modules=["q_proj", "v_proj"],
21             lora_dropout=0.05,
22         )
23         model = get_peft_model(model, lora_cfg)
24
25     training_args = TrainingArguments(
26         output_dir=output_dir,
27         num_train_epochs=5,
28         per_device_train_batch_size=4,
29         gradient_accumulation_steps=4,
30         learning_rate=2e-5,
31         lr_scheduler_type="cosine",
32         warmup_ratio=0.05,
33         bf16=True,
34         logging_steps=50,
35         save_strategy="epoch",
36         evaluation_strategy="epoch",
37     )
38     trainer = SFTTrainer(
39         model=model,
40         tokenizer=tokenizer,
41         train_dataset=load_dataset(dataset_path)["train"],
42         eval_dataset=load_dataset(dataset_path)["validation"],
43         args=training_args,
44     )
45     trainer.train()

```

Листинг Б.3 – Скрипт запуска обучения на PyTorch (фрагмент)


```

1 import re
2 import json
3 from dataclasses import dataclass
4
5 @dataclass
6 class StructureMetrics:
7     has_relevant_contexts: bool
8     has_analysis: bool
9     has_label: bool
10    label_valid: bool
11    json_parseable: bool
12
13    @property
14    def score(self) -> float:
15        fields = [
16            self.has_relevant_contexts,
17            self.has_analysis,
18            self.has_label,
19            self.label_valid,
20            self.json_parseable,
21        ]
22        return sum(fields) / len(fields)
23
24 def evaluate_structure(response: str) ->
    StructureMetrics:
25     """Оценивает
        структурное соответствие ответа ожидаемому формату
        """
26     try:
27         data = json.loads(response)
28         parseable = True
29     except json.JSONDecodeError:
30         return StructureMetrics(False, False, False,
            False, False)

```

```
31 label = data.get("label")
32
33 return StructureMetrics(
34     has_relevant_contexts=isinstance(data.get("
35         relevant_contexts"), list),
36     has_analysis=bool(data.get("analysis", "").
37         strip()),
38     has_label="label" in data,
39     label_valid=label in (0, 1, 2),
40     json_parseable=parseable,
```

Листинг Б.4 – Regex-метрики оценки структуры ответа

ПРИЛОЖЕНИЕ В. СКРИНШОТЫ ИНТЕРФЕЙСА ПЛАТФОРМЫ РАЗМЕТКИ

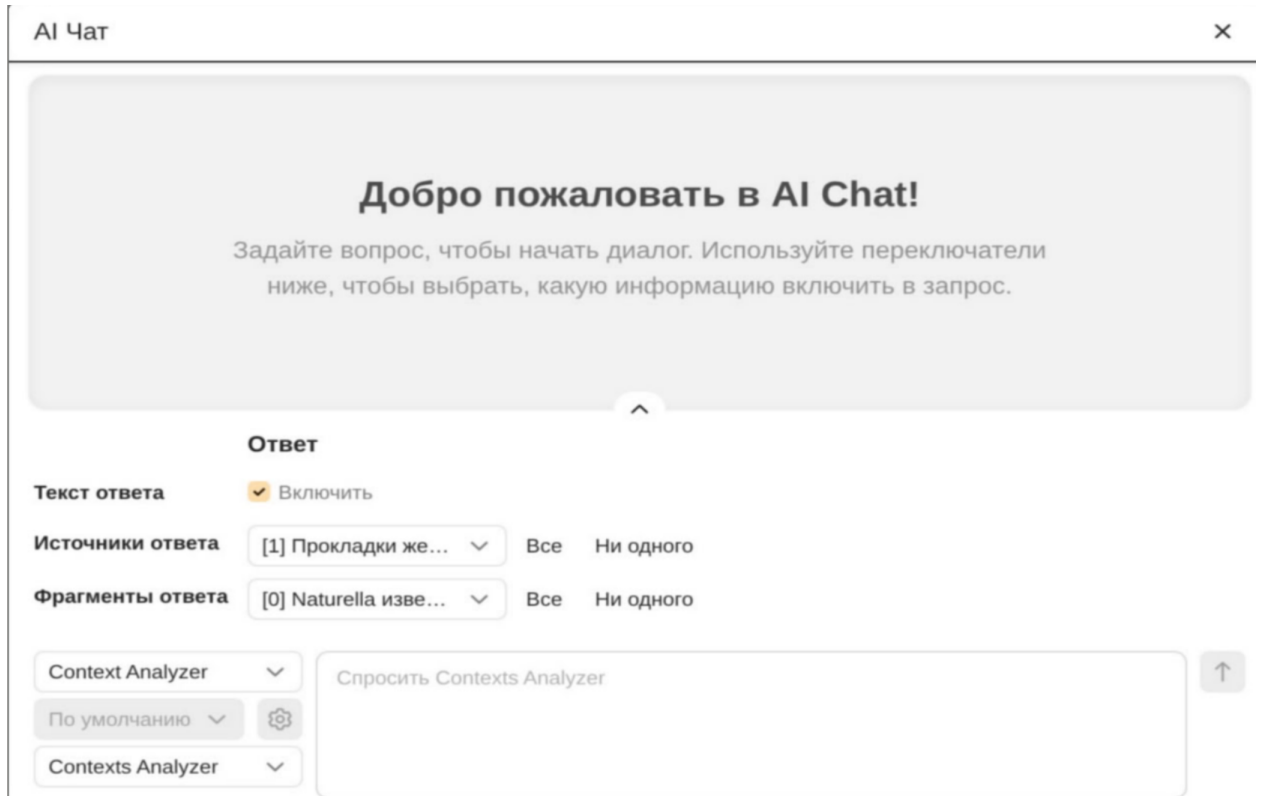


Рисунок В.1 – Диалоговое окно ассистента разметчика (скриншот интерфейса)

На рисунке В.1 приведён скриншот диалогового окна инструмента анализа информационных контекстов в интерфейсе платформы текстовой разметки (ООО «Яндекс.Технологии»): видна форма запроса, область информационных контекстов и структурированный ответ ассистента.